Explicación del código – Proyecto de productos con React, Express y MySQL

Samuel Santiago López

Análisis y Desarrollo de Software

Frontend: C3312932

Yoly Alexandra Guevara Muñoz

6 de noviembre del 2026

Introducción

En este documento explico cómo funciona mi proyecto de productos, que está dividido en dos partes que trabajan juntas. La primera es el **frontend**, que es la parte visual que ve el usuario en el navegador; la hice con React y Vite, y usa React Router para pasar de una página a otra sin recargar. La segunda es el **backend**, que es el servidor que recibe los pedidos del frontend, guarda y busca la información en una base de datos MySQL y devuelve las respuestas.

La idea es que el frontend nunca habla directamente con la base de datos: siempre le pide las cosas al backend, y el backend es quien se encarga de MySQL. Para comprobar que todo funciona, probé primero el servidor con Postman y después lo conecté con la parte visual. También dejé espacios para pegar las capturas de pantalla de cada paso.

Preparación de los proyectos

Trabajé con dos proyectos separados, cada uno en su propia carpeta y abierto en Visual Studio Code. Separarlos hace que el trabajo quede ordenado y que cada parte se pueda cambiar sin dañar la otra. Los comandos que se usan para prepararlos son estos:

Proyecto del frontend

- Crear el proyecto con React y Vite: **npm create vite@latest navegacion -- --template react**
- Entrar a la carpeta del proyecto: **cd navegacion**
- Instalar los programas que necesita el proyecto: **npm install**
- Instalar la herramienta para moverse entre páginas: **npm install**
- Abrir el proyecto en Visual Studio Code: **code .**

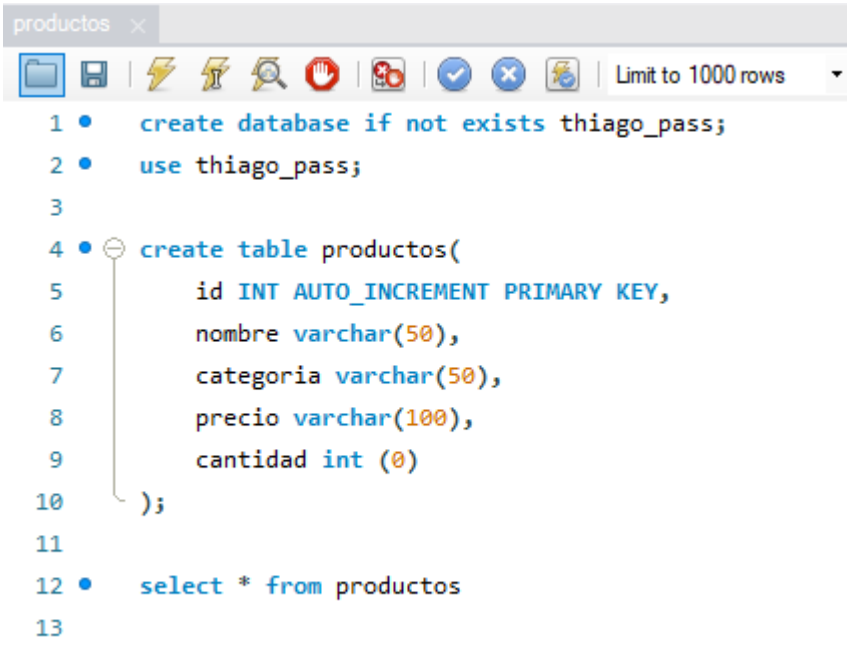
Proyecto del backend

- Crear una carpeta para el servidor y entrar en ella.
- Iniciar el proyecto con Node.js: **npm init -y**
- Instalar lo que necesita el servidor: **npm install express mysql2 cors**
- Abrir el proyecto en Visual Studio Code: **code .**

Base de datos MySQL y pruebas en Postman

El script de la base de datos

Antes de encender el servidor creé la base de datos y la tabla donde se guardan los productos. Este es el script que escribí en MySQL Workbench (archivo **productos.sql**):



Qué hace cada parte

- **create database if not exists thiago_pass;** crea la base de datos llamada thiago_pass, pero solo si todavía no existe. Así no da error si la ejecuto dos veces.
- **use thiago_pass;** le dice a MySQL que trabaje dentro de esa base de datos.
- **create table productos:** crea la tabla, que funciona como una hoja de Excel: cada producto es una fila y cada dato es una columna.
- **select * from productos** muestra todos los productos guardados. Lo uso para comprobar que lo que mando desde Postman o desde la página sí quedó guardado.

-

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

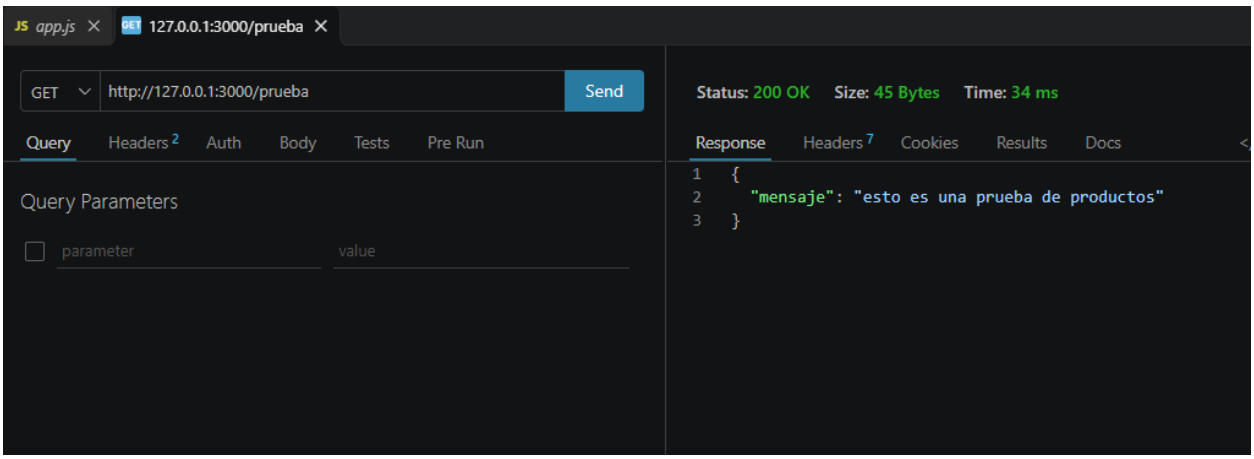
id	nombre	categoria	precio	cantidad
1	NULL	NULL	NULL	NULL
2	NULL	NULL	NULL	NULL
3	sssas	2	333	2
NULL	NULL	NULL	NULL	NULL

Probar la API en Postman

Antes de conectar la parte visual quise comprobar que el servidor funciona solo. Para eso uso Postman, una herramienta que permite enviar pedidos al servidor y ver qué responde. El servidor debe estar encendido (**npm run dev**) y escucha en el puerto 3000. Hice tres pruebas:

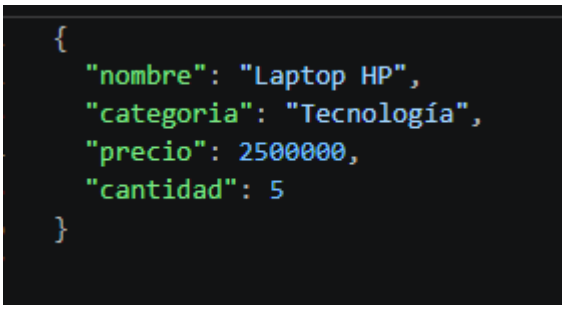
¿está encendido el servidor?

Pedido **GET** a **http://localhost:3000/prueba**. Debe responder con un mensaje que confirma que la API funciona.

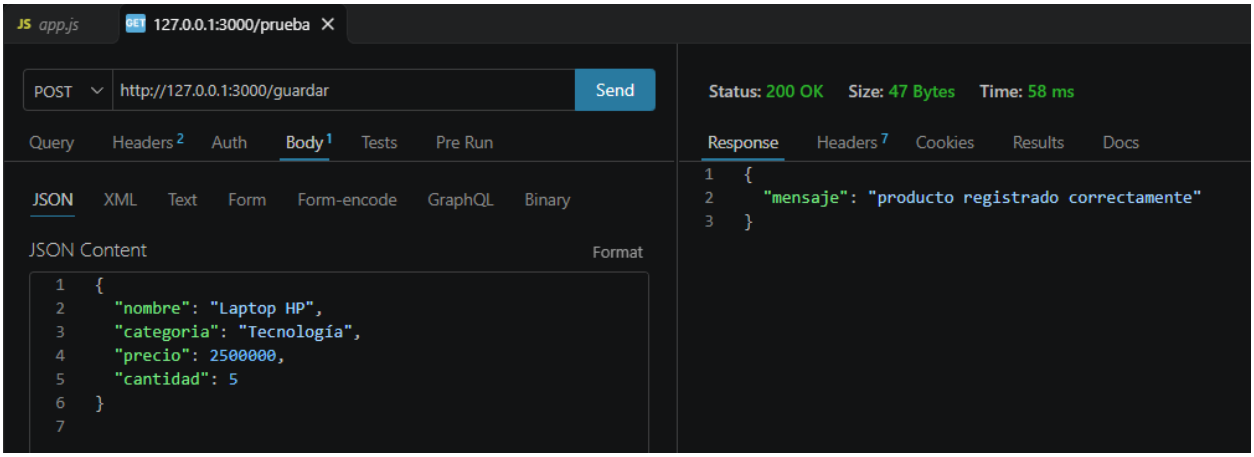


guardar un producto

Pedido **POST** a **http://localhost:3000/guardar**. En la pestaña **Headers** agrego **Content-Type** con el valor **application/json**, que le avisa al servidor que le voy a enviar datos en formato JSON. En la pestaña **Body** elijo la opción *raw* con formato JSON y escribo un producto de prueba:

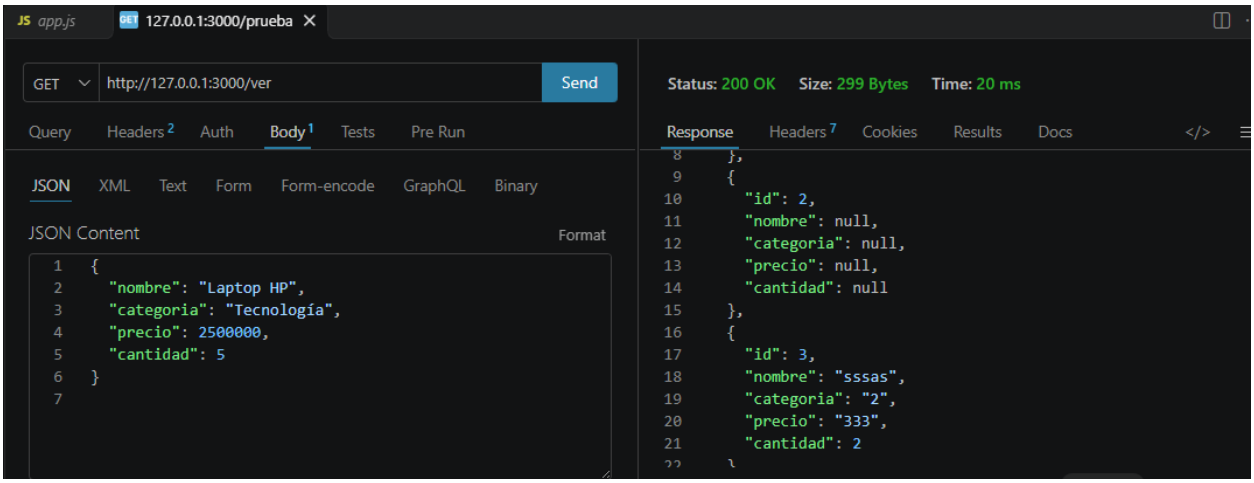


Al enviar, el servidor debe responder que el producto fue registrado correctamente.



ver los productos guardados

Pedido **GET** a **http://localhost:3000/ver**. Debe devolver la lista de productos, incluyendo el que acabo de guardar. Así compruebo que los datos quedaron guardados de verdad en MySQL y no solo en la memoria del servidor.



id	nombre	categoria	precio	cantidad
1	NULL	NULL	NULL	NULL
2	NULL	NULL	NULL	NULL
3	sssas	2	333	2
4	Laptop...	Tecnología	2500...	5
NULL	NULL	NULL	NULL	NULL

El frontend con React (archivos de configuración)

El frontend es la parte que se ve en el navegador. Lo creé con **React**, una herramienta para construir páginas por pedazos llamados componentes, y con **Vite**, que es el programa que enciende el proyecto y lo muestra rápido mientras programo. En esta fase explico los archivos de configuración del proyecto, que son los que dicen cómo funciona todo.

El archivo package.json

Es la ficha del proyecto: dice cómo se llama, qué comandos tiene y qué programas extra usa.



El backend con Node.js y Express

- **npm start** enciende el servidor normalmente, y **npm run dev** lo enciende en modo vigilancia: se reinicia solo cada vez que guardo un cambio.
- **"type": "module"** permite escribir **import** en lugar de **require**, que es la forma moderna de traer librerías.
- **express:** crea el servidor y las rutas. **mysql2:** conecta con MySQL. **cors:** deja que el frontend (que está en otro puerto) pueda hablar con el servidor. **dotenv:** sirve para guardar datos privados, como la contraseña, en un archivo aparte.

Un detalle: dotenv está instalado pero, como se ve más abajo, todavía no se usa en el código. Lo explico en la fase de mejoras.

El código del servidor (app.js), paso a paso

traer las librerías y preparar el servidor

```
import express from "express"
import cors from "cors"
import mysql from "mysql2"

const app = express()
const PORT = 3000
```

Las tres primeras líneas traen las librerías instaladas. Luego **app = express()** crea el servidor y **PORT = 3000** define el puerto por donde va a recibir los pedidos. Por eso mi API se abre en **http://localhost:3000**.

los dos ayudantes del servidor

```
app.use(express.json())
app.use(cors())
```

- **express.json()** hace que el servidor entienda los datos que llegan en formato JSON. Sin esta línea, lo que envío desde Postman o desde React llegaría vacío.
- **cors()** permite que el frontend, que corre en otro puerto (normalmente el 5173 con Vite), pueda pedirle cosas al servidor. Sin esto, el navegador bloquea el pedido por seguridad.

conectarse a MySQL

```
// conexion a la base de datos
const conectar = mysql.createConnection({
    host: "127.0.0.1",
    user: "root",
    password: "Almendra",
    database: "thiago_pass",
    port: 3306
})

conectar.connect((error) => {
    if (error) {
        console.log("error al conectar la base de datos", error.message)
    } else {
        console.log("conexion exitosa")
    }
})
```

Aquí le doy al servidor los datos para entrar a MySQL, como la dirección y las llaves de una casa:

- **host:** dónde está MySQL. **127.0.0.1** significa “en este mismo computador”.
- **user y password:** el usuario y la contraseña.
- **database:** la base de datos que se va a usar, en este caso **thiago_pass**.
- **port:** la puerta de MySQL, normalmente la 3306.

Después, **conectar.connect** intenta entrar. Si falla, escribe en la consola el motivo del error (**error.message**); si todo va bien, escribe que la conexión fue exitosa. Esto me deja saber al instante si el servidor y MySQL se están entendiendo.

la ruta de prueba

```
//prueba
app.get("/prueba", (req, res) => {
    res.json({ mensaje: "esto es una prueba de productos" })
})
```

Solo sirve para comprobar que el servidor está encendido. No usa la base de datos. Cuando entro a **http://localhost:3000/prueba**, responde con un mensaje. En el código, **req** es el pedido que llega y **res** es la respuesta que se devuelve.

guardar un producto

```
app.post("/guardar", (req, res) => {
    const { nombre, categoria, precio, cantidad } = req.body
    conectar.query(
        "INSERT INTO productos(nombre,categoria,precio,cantidad) VALUES (?,?,?,?)",
        [nombre, categoria, precio, cantidad],
        (error, resultado) => {
            if (error) {
                return res.json({ mensaje: "error al registrar el producto" })
            } else {
                return res.json({ mensaje: "producto registrado correctamente" })
            }
        }
    )
})
```

1. Saca del pedido los cuatro datos del producto: nombre, categoría, precio y cantidad.
2. Le pide a MySQL que guarde una fila nueva. Los signos de interrogación **?** son espacios que se llenan con los datos del arreglo que viene después. Hacerlo así protege la base de datos, porque evita que alguien “cuele” instrucciones peligrosas dentro de los datos (un ataque llamado inyección SQL).

3. Cuando MySQL termina, se ejecuta la función del final: si hubo un error, responde con un mensaje de error; si todo salió bien, responde que el producto quedó registrado.

ver los productos

Le pide a MySQL todos los productos (**SELECT * FROM productos** significa “tráeme todo lo de la tabla productos”) y los devuelve en una lista. Esta es la ruta que usa la página de React para llenar la tabla. Si algo falla, avisa con un mensaje en lugar de quedarse sin responder.

```
app.get("/ver", (req, res) => {
    conectar.query("SELECT * FROM productos", (error, resultado) => {
        if (error) {
            return res.json({ mensaje: "no se encuentran los datos" })
        } else {
            return res.json(resultado)
        }
    })
```

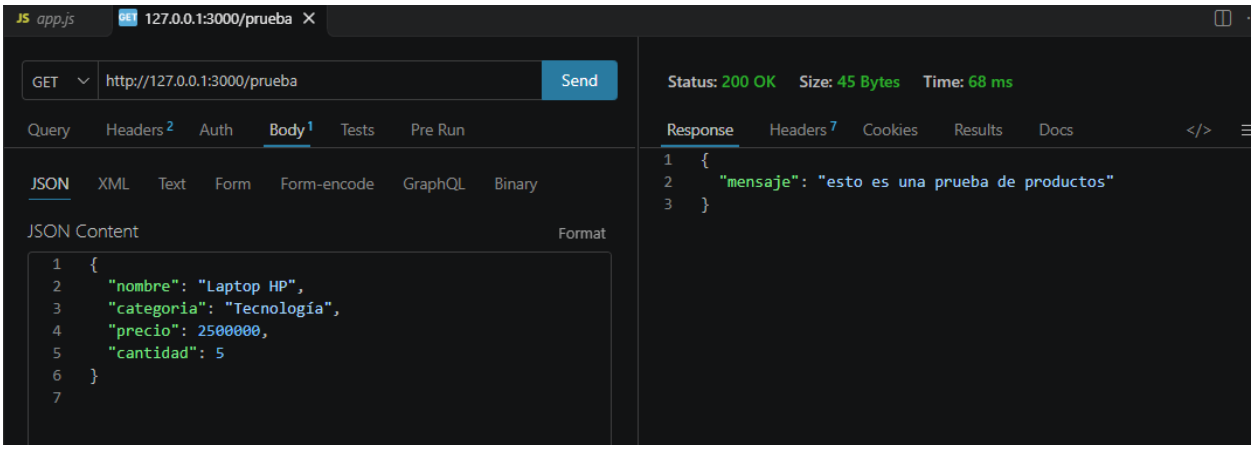
encender el servidor

app.listen deja el servidor escuchando pedidos en el puerto 3000 y escribe un mensaje en la consola para avisarme que ya está listo.

```
app.listen(PORT, () => {
    console.log("servidor en ejecucion en el puerto " + PORT)
})
```

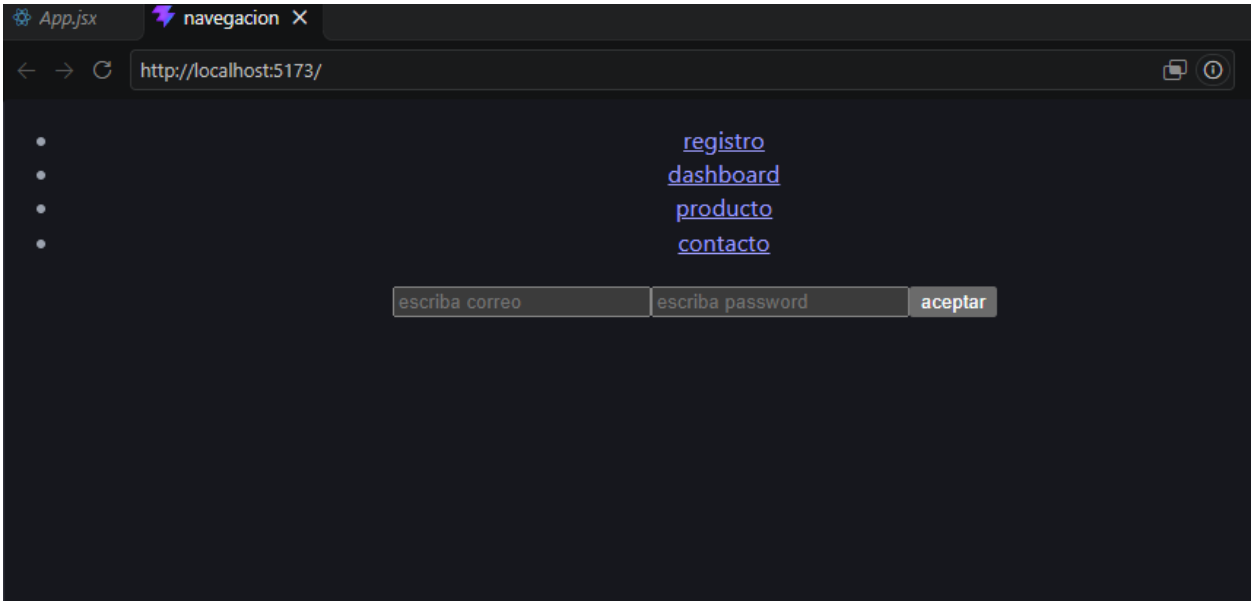
Resumen de las rutas

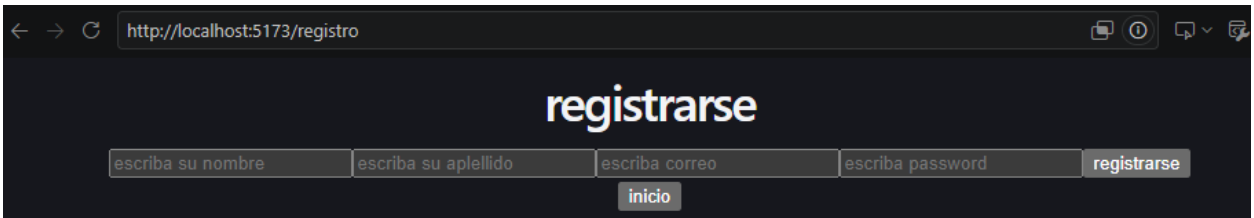
- **GET /prueba:** comprueba que el servidor funciona.
- **POST /guardar:** guarda un producto nuevo en MySQL.
- **GET /ver:** devuelve la lista de todos los productos guardados.

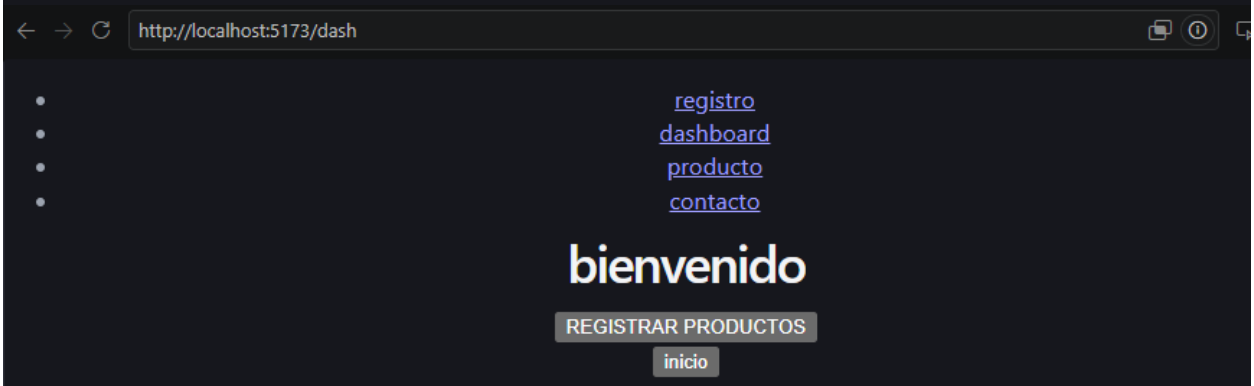


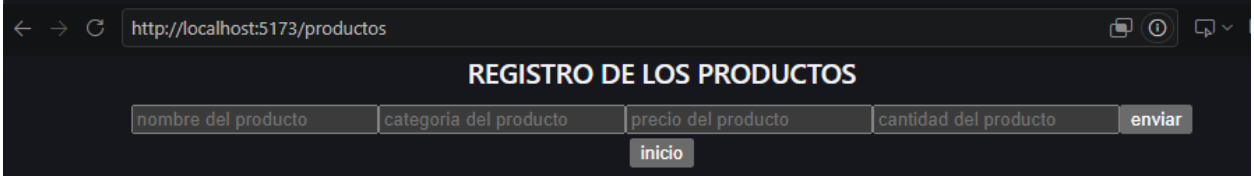

```
servidor en ejecucion en el puerto 3000
conexion exitosa
```

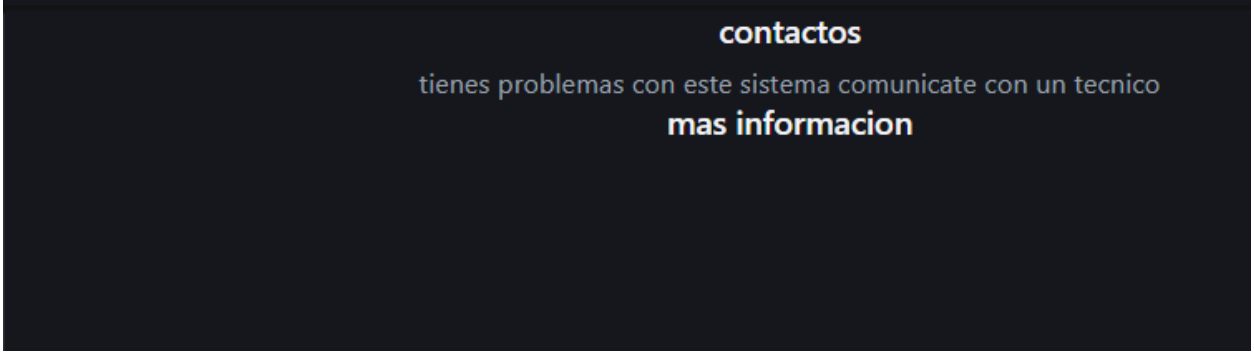
Pruebas del frontend en el navegador

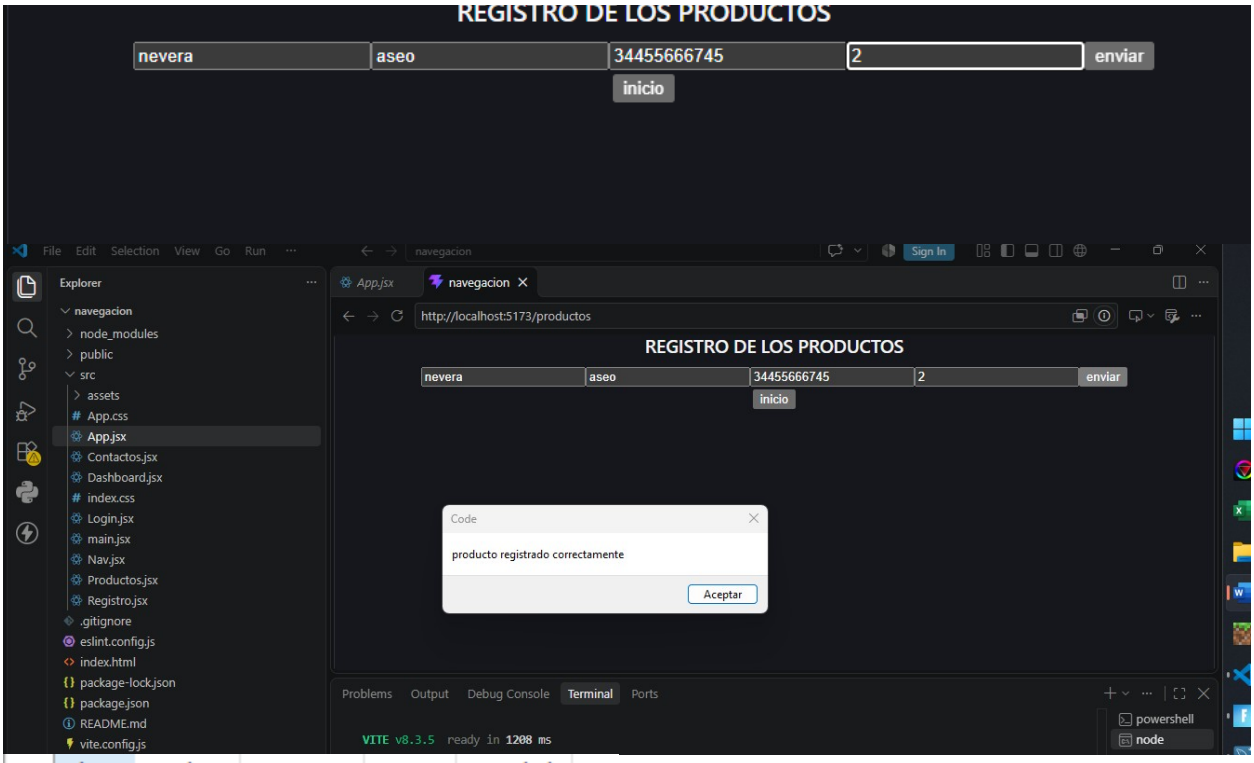



http://localhost:5173/registro
registrarse
escriba su nombre
escriba su aplellido
escriba correo
escriba password
registrarse
inicio


http://localhost:5173/dash
registro
dashboard
producto
contacto
bienvenido
REGISTRAR PRODUCTOS
inicio


http://localhost:5173/productos
REGISTRO DE LOS PRODUCTOS
nombre del producto
categoria del producto
precio del producto
cantidad del producto
enviar
inicio


contactos
tienes problemas con este sistema comunicate con un tecnico
mas informacion



id	nombre	categoria	precio	cantidad
1	NULL	NULL	NULL	NULL
2	NULL	NULL	NULL	NULL
3	sssas	2	333	2
4	Laptop...	Tecnología	2500...	5
5	nevera	aseo	3445...	2
NULL	NULL	NULL	NULL	NULL

Palabras clave explicadas fácil

- **Frontend:** la parte del programa que ve y usa la persona en el navegador.
- **Backend:** la parte que trabaja “detrás”: recibe los pedidos, usa la base de datos y responde.
- **API:** el conjunto de rutas del backend a las que el frontend le hace pedidos.
- **GET y POST:** GET pide información sin cambiar nada; POST envía información nueva para guardarla.
- **JSON:** formato de texto sencillo para intercambiar datos, parecido a una ficha con nombre y valor.
- **CORS:** permiso que le da el servidor a páginas de otros puertos o dominios para que puedan pedirle datos. Sin él, el navegador bloquea el pedido por seguridad.

- **Variable de entorno:** dato privado (como una contraseña) que se guarda fuera del código, en el archivo **.env**.
- **Consulta parametrizada:** forma segura de enviar datos a la base de datos usando **?**, para que nadie pueda colar instrucciones peligrosas.
- **Código de estado:** número que indica cómo salió el pedido: 200 todo bien, 201 creado, 400 datos mal enviados, 500 error del servidor.

Conclusión

Con este proyecto entendí cómo trabajan juntas las tres partes de una aplicación: la página que ve el usuario (React), el servidor que recibe los pedidos y los procesa (Express) y la base de datos que guarda la información de forma permanente (MySQL). También entendí por qué el frontend nunca debe conectarse directo a la base de datos: el servidor es el que protege la información y valida lo que entra.